

Engine Technical Specification

Local Audio Engine (Production)

1. Scope

Defines architecture, data flow, timing model, and modules for the local audio engine responsible for realtime audio processing and scoring.

Language: Rust (preferred) Runtime: native process IO: audio device + WebSocket server

2. Core Principles

1. Engine is source of truth for time and scoring
 2. Realtime safe processing (no blocking in audio thread)
 3. Deterministic session runtime
 4. Clear separation: audio thread vs logic thread
-

3. High-Level Architecture

Threads: - Audio Thread (RT) - Processing Thread - Network Thread

Data flow: Audio → Buffer → DSP → Detection → Event Queue → Scoring → WS Output

4. Audio Subsystem

4.1 Configuration

- Sample rate: 44100 / 48000
- Buffer size: 128-512
- Channels: mono

4.2 Audio Thread (RT)

Responsibilities: - capture samples - push to lock-free ring buffer

Constraints: - no allocations - no locks - no logging

5. Processing Pipeline

5.1 Preprocessing

- high-pass filter (70–100 Hz)
- noise gate
- normalization (lightweight)

5.2 Onset Detection

Input: sample frames Output: onset event (timestamp)

Algorithm (MVP): - energy delta threshold

Output struct:

```
struct OnsetEvent { time_ms: u64 }
```

5.3 Pitch Detection

Input: windowed buffer Output:

```
struct PitchResult {  
    freq_hz: f32,  
    note: Note,  
    confidence: f32  
}
```

MVP: autocorrelation/YIN-like

5.4 Event Builder

Combine onset + pitch:

```
struct NoteEvent {  
    onset_ms: u64,  
    freq_hz: f32,  
    note: Note,  
    confidence: f32  
}
```

Filter: confidence threshold

6. Session Runtime

6.1 Clock

- monotonic clock (engine local)
- sessionStartTime captured at start

6.2 Note Map

```
struct ExpectedNote {  
    time_ms: u64,  
    note: Note  
}
```

6.3 Matching

For each NoteEvent: - find nearest ExpectedNote - compute offset = eventTime - expectedTime

7. Scoring

7.1 Windows

- perfect: <=30 ms
- good: <=80 ms
- else: miss

7.2 Output

```
enum Judgement { Perfect, Good, Miss }  
  
struct ScoreEvent {  
    judgement: Judgement,  
    offset_ms: i32  
}
```

7.3 Aggregation

- combo
 - accuracy
 - score
-

8. Communication Layer

8.1 WebSocket Server

- async runtime

- non-blocking

8.2 Event Queue

- lock-free queue between processing and network thread

8.3 Output Messages

- note_event
 - score_update
 - session_result
-

9. Error Handling

Categories: - device - audio init - runtime - protocol

All errors must: - be logged - be emitted via protocol

10. Diagnostics

Expose: - input level - latency estimate - buffer health

11. Performance Constraints

- end-to-end detection latency target: <50 ms
 - CPU usage: minimal per frame
 - no GC pauses (Rust)
-

12. Definition of Done

Engine is production-ready when: - stable audio capture - reliable onset detection - stable pitch detection for single notes - deterministic scoring - no crashes during long sessions - WebSocket integration stable